

SCORED CONSISTENCY DISTILLATION OF SD3.5-MEDIUM WITH A DECODE-FREE PHOTOGRAPHIC REWARD

Anonymous authors

Paper under double-blind review

1 METHOD

We distil SD3.5-Medium, a 2.5B-parameter rectified-flow transformer, into a 4-step student. The method has two parts. First, for every training caption the frozen teacher produces several candidate generations, and a scorer that reads a reference photograph of the caption picks one of them; the student is trained only on the picked trajectory. Second, a differentiable, decode-free photographic reward term pulls the student’s own predictions toward the same photograph during training, computed by a small trained network rather than by decoding through the VAE. Both parts use the same underlying quantity, the DINOv2 similarity of an image to the photograph the caption describes, and neither reads the caption text or any language model.

1.1 PRELIMINARIES: RECTIFIED-FLOW DISTILLATION

The teacher is a rectified-flow model with frozen weights θ_T , operating in the D -dimensional latent space of a frozen VAE ($D=65,536$ at 512×512 pixels: a $16\times 64\times 64$ latent). A clean latent $x_0 \in \mathbb{R}^D$ and Gaussian noise $\varepsilon \sim \mathcal{N}(0, I)$ define a straight path $z_\sigma = (1 - \sigma)x_0 + \sigma\varepsilon$, $\sigma \in [0, 1]$, and the network predicts the velocity $v = \varepsilon - x_0$ conditioned on the caption c . Sampling is Euler integration on a σ grid, $z_{k+1} = z_k + (\sigma_{k+1} - \sigma_k) v_\theta(z_k, \sigma_k, c)$, with classifier-free guidance $v^{\text{cfg}} = v_\theta(z, \sigma, \emptyset) + w(v_\theta(z, \sigma, c) - v_\theta(z, \sigma, \emptyset))$ at $w = 7$ for the teacher. The student v_θ is a full copy of the same transformer, initialised from θ_T , and is the only network that trains; the teacher, the text encoders and the VAE decoder Dec stay frozen throughout. There is no momentum target, no exponential moving average and no stop-gradient copy of the student anywhere in the recipe: every training target below is a deterministic function of the frozen teacher.

Consistency distillation trains the student to jump directly to the clean-latent estimate that the teacher reaches only after several further steps, so that a 4-step Euler rollout of the student reproduces an 8-step teacher trajectory. This is the sole mechanism that turns an 8-step teacher into a 4-step student; nothing below changes it. What varies is which trajectory the student is trained on, and what, besides matching that trajectory, the training loss asks the student’s own predictions to do.

1.2 CANDIDATE GENERATION AND PHOTOGRAPHIC SCORING

For every training caption c with reference photograph x^{ref} (its COCO source image), we draw $N = 4$ candidate teacher trajectories from four independent seeds and score each one against the photograph.

Candidates. Candidate j uses Gaussian noise $\varepsilon^{(j)}$ seeded deterministically from the caption index, and is an 8-step Euler rollout of the teacher at guidance $w = 7$:

$$\varepsilon^{(j)} \sim \mathcal{N}(0, I) \text{ (seeded by caption and } j), \quad z_0^{(j)}, \dots, z_8^{(j)} = \text{Euler}_{K=8}(\varepsilon^{(j)}; v_T^{\text{cfg}}, c). \quad (1)$$

Only the seed, not the trajectory, needs to be stored: any candidate can be re-rolled exactly from its seed whenever it is needed again, at scoring time and later at training time.

Scoring. Each candidate is decoded, $I_j = \text{Dec}(z_8^{(j)})$, and embedded with a frozen DINOv2-B vision transformer. We drop the CLS token, mean-pool the $P=256$ patch tokens and ℓ_2 -normalise:

$$u(I) = \frac{\frac{1}{P} \sum_{p=1}^P h_p(I)}{\left\| \frac{1}{P} \sum_{p=1}^P h_p(I) \right\|}, \quad S_j = \langle u(I_j), u(x^{\text{ref}}) \rangle. \quad (2)$$

$u(x^{\text{ref}})$ is computed once per caption. This scorer needs no caption text, no VLM and no learned preference model: it is a fixed, off-the-shelf self-supervised image encoder, applied identically to a generated candidate and to a real photograph.

Selection. We select the best candidate by hard argmax over the score,

$$j^* = \arg \max_{j \in \{1, \dots, N\}} S_j, \quad (3)$$

and every subsequent training visit of caption c trains only on trajectory j^* . Selection is computed once, offline, before training starts, and is never revisited: no gradient flows through the scorer, and the training loop simply re-rolls trajectory j^* from its stored seed. A random baseline replaces j^* with a uniform draw over $\{1, \dots, N\}$, stored so that every training seed sees the same draw; this isolates the effect of *which* trajectory a caption trains on from everything else in the recipe.

1.3 CONSISTENCY DISTILLATION LOSS

Let $z_0, \dots, z_8$ be the selected teacher trajectory. Training supervises a window of states $\mathcal{K} = \{3, 4, 5, 6, 7\}$ on the 8-step grid. For $k \in \mathcal{K}$, the teacher’s realised chord across its own step and the clean-latent estimate it implies are

$$\bar{v}_k = \frac{z_{k+1} - z_k}{\sigma_{k+1} - \sigma_k}, \quad \tilde{x}_0^{(k)} = z_k - \sigma_k \bar{v}_k, \quad (4)$$

which is exact algebra for a straight path. The student is placed at a *noisier* state than the one the target was read from: for each k we draw $\Delta_k \sim \mathcal{U}\{1, 2, 3\}$ independently and afresh on every visit, set $s = k - \Delta_k$, and the student forms its own clean-latent estimate with a single conditional forward pass and no guidance,

$$\hat{x}_0^{(s)} = z_s - \sigma_s v_\theta(z_s, \sigma_s, c). \quad (5)$$

The consistency loss is a pseudo-Huber distance in x_0 -space, averaged over the window:

$$\ell_c(\theta) = \frac{1}{|\mathcal{K}|} \sum_{k \in \mathcal{K}} \rho(\hat{x}_0^{(k-\Delta_k)} - \text{sg}[\tilde{x}_0^{(k)}]), \quad \rho(u) = \sqrt{\|u\|_2^2 + c_h^2} - c_h, \quad (6)$$

with $\text{sg}[\cdot]$ a stop-gradient and $c_h = 0.00054\sqrt{D} \approx 0.138$ a small constant scaled to the latent dimensionality (the pseudo-Huber loss behaves like an ℓ_2 distance below c_h and like an ℓ_1 distance above it). Because the student is trained with a single conditional pass against $w=7$ teacher trajectories, guidance is absorbed into its prediction: the student is sampled at $w=1$, with no classifier-free guidance at inference.

1.4 A DECODE-FREE PHOTOGRAPHIC REWARD

Selection alone only changes which of the teacher’s four trajectories the student is trained on; it never moves the student’s own predictions toward the photograph directly. The reward does, and it does so *without ever decoding a training image*: a small trained network P_ϕ maps a latent directly into the DINOv2 patch-mean space of Section 1.2, so a reward can be computed and differentiated on the student’s own latent prediction with no VAE decode, no DINOv2 forward pass, and no dependence on either at training time.

Projector network and its offline pretraining. $P_\phi : \mathbb{R}^{16 \times 64 \times 64} \rightarrow \mathbb{R}^{768}$ is a 6.8M-parameter convolutional encoder (five blocks, channel widths 64, 128, 256, 512, 512, the last four downsampling by 2, global-average-pooled) followed by a two-layer MLP head (512→1024→768) and an ℓ_2 -normalisation. It is pretrained once, offline, on 22,000 captions disjoint from any pool it will

later score: for each cached candidate z_j with true embedding $e_j = u(\text{Dec}(z_j))$ and true score $S_j = \langle e_j, u(x^{\text{ref}}) \rangle$,

$$\mathcal{L}_{\text{pre}}(\phi) = \text{mean}_j [1 - \langle P_\phi(z_j), e_j \rangle] + \text{KL}(\text{softmax}(S/\tau) \parallel \text{softmax}(\hat{S}/\tau)), \quad \hat{S}_j = \langle P_\phi(z_j), u(x^{\text{ref}}) \rangle, \quad (7)$$

$\tau=0.04$, 30 epochs, batch 32 captions (128 candidates), AdamW with a one-cycle schedule peaking at 3×10^{-4} ; the first term matches the embedding directly, the second preserves the within-caption ranking of the four candidates. The pretrained P_ϕ is then frozen and used to compute the training-time reward.

Reward. On every training update, the student’s own clean-latent estimates at the two least-noisy supervised states are scored by P_ϕ directly, with no decode:

$$r_c(\theta; \phi) = \frac{1}{2} \sum_{i=1}^2 \langle P_\phi(\hat{x}_0^{(s_i)}), u(x^{\text{ref}}) \rangle. \quad (8)$$

The full per-caption training objective is

$$\mathcal{L}(\theta) = \mathbb{E}_c[\ell_c(\theta)] - \lambda \mathbb{E}_c[r_c(\theta; \phi)], \quad (9)$$

combining Equations 6 and 8, with $\lambda=80$ (set, like the exact reward’s weight, so the raw reward gradient starts at about 20% of the raw consistency gradient’s norm; P_ϕ ’s outputs live on a different scale than the true DINOv2 embeddings, so this weight is not comparable in magnitude to a decode-based reward’s).

Periodic refresh. P_ϕ was pretrained on the teacher’s own candidates, not on a 4-step student’s evolving predictions, so its accuracy on the distribution it is actually scoring during training is not guaranteed. To address this, every 100 training updates P_ϕ is briefly unfrozen and fit to the true DINOv2 embedding of the student’s own recent predictions (decoded just for this refresh step, not for the reward itself), together with a replay term on a fresh batch of the original teacher-candidate data so it does not forget how to rank them:

$$\mathcal{L}_{\text{refresh}}(\phi) = \text{mean}[1 - \langle P_\phi(\hat{x}_0), u(\text{Dec}(\hat{x}_0)) \rangle] + \mathcal{L}_{\text{pre}}(\phi) \text{ on a replay batch}, \quad (10)$$

16 AdamW steps at learning rate 10^{-4} each time, after which P_ϕ is frozen again until the next refresh. This variant – more refresh steps per event than the default of 4 – is the best-scoring configuration of the projector-reward family we tested (Section 3.1).

1.5 TRAINING ALGORITHM

Algorithm 1 chooses one trajectory per caption, once, before training starts, and still decodes and scores candidates offline – that part of the pipeline is unchanged. Algorithm 2 pretrains the projector P_ϕ , once, before any student training. Algorithm 3 is the training loop itself: it never decodes a latent to compute the reward, only P_ϕ is called, and every 100 updates P_ϕ is briefly refit (lines 11–17).

At inference the student is sampled with 4 Euler steps on its own grid, $w=1$, with no guidance, no candidate generation and no projector: Algorithms 1, 2 and the refresh block of Algorithm 3 only apply at training time. The trained student is a single frozen network; nothing about candidate pools, scorers, the projector or the reward survives into the deployed model.

2 EXPERIMENTAL SETUP

Data. Training captions come from COCO `train2017`, each paired with the photograph it was originally written for; that photograph is used only by the scorer and the reward, and never enters the student’s training input. Two pools are used: a **3k pool** of 3,000 captions drawn once with a fixed seed, used for every multi-seed comparison, and a **118k pool** of 113,948 captions, used to test whether the recipe holds at scale. All evaluation uses a separate, sealed pool built once with a fixed seed; evaluation references and fidelity targets come from COCO `val2017`, so no evaluation caption or photograph is seen in training.

Algorithm 1 Choose a trajectory for every caption (once, before training)

Require: training captions with reference photographs x^{ref} ; frozen teacher v_T ; frozen decoder Dec; frozen DINOv2 scorer $u(\cdot)$

- 1: **for** each training caption c **do**
- 2: **for** $j = 1, \dots, N$, with $N=4$ **do**
- 3: draw noise $\varepsilon^{(j)} \sim \mathcal{N}(0, I)$ from a seed fixed by (c, j)
- 4: roll out the teacher for 8 Euler steps at guidance $w=7$: $z_0^{(j)}, \dots, z_8^{(j)} = \text{Euler}_8(\varepsilon^{(j)}; v_T, c)$
- 5: decode and score: $I_j \leftarrow \text{Dec}(z_8^{(j)})$, $S_j \leftarrow \langle u(I_j), u(x^{\text{ref}}) \rangle$ ▷ Equation 2
- 6: **end for**
- 7: $j^* \leftarrow \arg \max_j S_j$ ▷ Equation 3
- 8: keep only $(c, \text{the seed of } j^*, z_8^{(1)}, \dots, z_8^{(4)}, e_1, \dots, e_4 = u(I_1), \dots, u(I_4))$ ▷ terminal latents + true embeddings kept as pretraining/replay data
- 9: **end for**

Algorithm 2 Pretrain the projector P_ϕ (once, before any student training)

Require: 22,000 captions' cached candidates from Algorithm 1 (z_j , true embedding e_j , true score S_j), disjoint from any pool P_ϕ will later score; $\tau=0.04$

- 1: initialise P_ϕ : 5-block conv encoder (widths 64,128,256,512,512) + MLP head (512→1024→768) + ℓ_2 -normalise
- 2: **for** 30 epochs, batches of 32 captions (128 candidates) **do**
- 3: $\hat{e}_j \leftarrow P_\phi(z_j)$; $\hat{S}_j \leftarrow \langle \hat{e}_j, u(x^{\text{ref}}) \rangle$
- 4: $\mathcal{L}_{\text{pre}} \leftarrow \text{mean}_j [1 - \langle \hat{e}_j, e_j \rangle] + \text{KL}(\text{softmax}(S/\tau) \parallel \text{softmax}(\hat{S}/\tau))$ ▷ Equation 7
- 5: AdamW step on ϕ (one-cycle schedule, peak lr 3×10^{-4})
- 6: **end for**
- 7: freeze ϕ ; this is the P_ϕ used by Algorithm 3 below

Training configurations. Table 1 gives both configurations. Both use AdamW, global gradient-norm clipping at 1, fp32 master weights under bf16 autocast, and a frozen teacher, text encoders and VAE. The 118k configuration uses 4 GPUs in data parallel, one caption per GPU per step; its reported student is the uniform weight average of the last five saved checkpoints, which is necessary because consecutive checkpoints otherwise differ by up to 0.035 CompBench under this schedule.

Evaluation. We report alignment with T2I-CompBench (8 categories: color, shape, texture, 2D-spatial, 3D-spatial, numeracy, non-spatial, complex; one image per prompt unless stated, and the official ten-images-per-prompt protocol where noted) and GenEval2 (object, count, attribute, position and verb skills, atom counts 3 to 10, scored as the geometric mean over atoms). We report fidelity with FID, CMMD, precision and recall against 5,000 held-out COCO val2017 photographs. Every arm and every seed is evaluated on exactly the same evaluation pool. Unless stated, all reported numbers are for a student sampled with 4 Euler steps, $w=1$.

3 RESULTS

3.1 ALIGNMENT

Table 2 compares three training recipes on the 3k pool, matched in every other respect: random trajectory selection with no reward (the naive baseline), argmax selection with no reward, and argmax selection with the decode-free projector reward, refreshed every 100 updates at 16 steps per refresh (our full method, Section 1.4). Selection is the effect that is doing the work here: it alone is worth $+0.0117$ CompBench over the baseline ($p = 0.011$); adding the reward on top of selection is worth a further $+0.0014 \pm 0.0035$ ($p = 0.57$), not resolved from zero at three seeds, and moves GenEval2 in the wrong direction (-0.19 ± 1.24). Against the naive baseline, the full method is $+0.0149 \pm 0.0017$ CompBench ($p = 0.004$) and $+1.00 \pm 0.91$ GenEval2 – a real gain over doing

Algorithm 3 Train the student (one optimiser step; repeat until the schedule ends)

Require: student v_θ (trainable, initialised from θ_T); frozen teacher v_T ; frozen decoder Dec; frozen DINOv2 scorer $u(\cdot)$; pretrained projector P_ϕ (Algorithm 2); replay buffer of teacher-candidate data (Algorithm 1); window $\mathcal{K} = \{3, \dots, 7\}$; reward weight $\lambda=80$

- 1: pick a caption c ; regenerate its kept trajectory $z_0, \dots, z_8$ from the stored seed
- 2: **for** $k \in \mathcal{K}$ **do** ▷ 5 supervised noise levels, batched into one student forward pass
- 3: draw $\Delta_k \sim \mathcal{U}\{1, 2, 3\}$ afresh; let $s = k - \Delta_k$ (a noisier state than k)
- 4: student’s clean-latent guess: $\hat{x}_0^{(s)} \leftarrow z_s - \sigma_s v_\theta(z_s, \sigma_s, c)$ ▷ one conditional pass, no guidance
- 5: teacher’s target, one step ahead, stop-gradient: $\tilde{x}_0^{(k)} \leftarrow \text{sg} [z_k - \sigma_k (z_{k+1} - z_k) / (\sigma_{k+1} - \sigma_k)]$
- 6: **end for**
- 7: **consistency loss:** $\ell_c \leftarrow \frac{1}{|\mathcal{K}|} \sum_{k \in \mathcal{K}} \left(\sqrt{\|\hat{x}_0^{(k-\Delta_k)} - \tilde{x}_0^{(k)}\|_2^2 + c_h^2} - c_h \right)$ ▷ pseudo-Huber,
- 8: $c_h = 0.00054\sqrt{D}$
- 9: **decode-free reward:** $r_c \leftarrow \frac{1}{2} \sum_{i=1}^2 \langle P_\phi(\hat{x}_0^{(s_i)}), u(x^{\text{ref}}) \rangle$ ▷ Equation 8; no Dec call, gradient flows back into θ only
- 10: $g \leftarrow \nabla_\theta [\ell_c - \lambda r_c]$; clip to global norm 1; $\theta \leftarrow \text{AdamW}(\theta, g)$ ▷ ϕ is NOT updated here
- 11: **if** this update’s index is a multiple of 100 **then** ▷ periodic refresh
- 12: decode a batch of the student’s recent $\hat{x}_0$ predictions, embed with $u(\text{Dec}(\cdot))$: this IS a decode, but only for refreshing ϕ , not for the reward gradient into θ above
- 13: sample a fresh replay batch from Algorithm 1’s stored candidates
- 14: **for** 16 steps **do** ▷ the best-scoring configuration; the default is 4 steps
- 15: AdamW step on ϕ (lr 10^{-4}) minimising $\mathcal{L}_{\text{refresh}}$ (Equation 10)
- 16: **end for**
- 17: freeze ϕ again
- 18: **end if**

Table 1: Training configurations. Both use identical code, differing only in scale.

	3k configuration	118k configuration
Captions	3,000	113,948
GPUs	1	4, data-parallel, one caption per GPU per step
Updates	6,000 (2 passes)	56,974 (2 passes)
Training seeds	3 – 5	3
Learning rate	10^{-5}	2×10^{-5}
Warm-up	300 steps	1,000 steps
Reward weight λ	80	– (Section 3.3)
Projector refresh	every 100 updates, 16 steps	–
Reported weights	final step	uniform average, last 5 checkpoints

nothing, but Section 3.2 shows it is attributable to selection, not to the reward the method adds on top of it.

For context: a differentiable reward that decodes the prediction through the real VAE instead of using the projector – otherwise the identical recipe – reaches 0.4923 CompBench, +0.0055 over argmax alone ($p = 0.004$), a real reward effect the projector version does not show. We report that comparison in full in Section 3.2, since it is the most direct evidence for why the projector’s refresh mechanism does not close the gap to decoding.

Per-category breakdown. Table 3 breaks the CompBench gain down by category against argmax alone, since that is the comparison the reward is meant to move. The picture matches the headline number: no category shows a resolved gain, and two move (mildly) in the wrong direction.

Table 2: Alignment on the 3k pool, averaged checkpoints, one image per prompt. Naive and argmax: five training seeds; the projector-reward row: three seeds (its own campaign). Deltas are seed-paired against the row above, mean $\pm$ standard deviation over seeds.

Recipe	CompBench	Δ	GenEval2 ($\times 100$)	Δ
Random selection, no reward (baseline)	0.4751 ± 0.0032	–	22.47 ± 0.68	–
Argmax selection, no reward	0.4868 ± 0.0031	$+0.0117 \pm 0.0058$ ($p=0.011$)	23.92 ± 1.29	$+1.46 \pm 1.38$
Argmax selection + projector reward, refreshed (ours)	0.4887 ± 0.0025	$+0.0014 \pm 0.0035$ ($p=0.57$)	23.54 ± 1.56	-0.19 ± 1.24
Ours vs. baseline	$+0.0149 \pm 0.0017$ CompBench ($p=0.004$),		$+1.00 \pm 0.91$ GenEval2	

Table 3: CompBench by category against argmax alone, 3k pool, three seeds, one image per prompt, averaged checkpoints. No category is resolved from zero once eight comparisons are considered.

Category	Argmax, no reward	Ours	Δ (seed-paired)
Color	0.8186 ± 0.0062	0.8284 ± 0.0055	$+0.0099 \pm 0.0038$ ($p=0.047$)
Shape	0.5610 ± 0.0250	0.5613 ± 0.0175	$+0.0002 \pm 0.0080$ ($p=0.963$)
Texture	0.7301 ± 0.0056	0.7294 ± 0.0067	-0.0007 ± 0.0033 ($p=0.756$)
2D-spatial	0.2353 ± 0.0142	0.2364 ± 0.0158	$+0.0011 \pm 0.0040$ ($p=0.673$)
3D-spatial	0.3157 ± 0.0133	0.3142 ± 0.0180	-0.0015 ± 0.0109 ($p=0.830$)
Numeracy	0.5466 ± 0.0357	0.5484 ± 0.0245	$+0.0018 \pm 0.0153$ ($p=0.857$)
Non-spatial	0.3112 ± 0.0009	0.3112 ± 0.0006	-0.0000 ± 0.0003 ($p=0.898$)
Complex	0.3799 ± 0.0037	0.3801 ± 0.0035	$+0.0002 \pm 0.0020$ ($p=0.904$)
Mean	0.4873	0.4887	$+0.0014 \pm 0.0035$ ($p=0.565$)

GenEval2 breakdown. Table 4 breaks GenEval2 down by skill and by the number of atomic sub-conditions a prompt has (3 to 10), against argmax alone. As with CompBench, the picture is flat: no skill and no atom-count bucket shows a resolved gain.

3.2 COMPARISON TO A DECODE-BASED REWARD, AND WHY THE PROJECTOR REWARD DOES NOT HELP

Table 5 repeats the same recipe with the only change being how the reward is computed: P_ϕ (Section 1.4, no decode) against $u \circ \text{Dec}$, the same reward formula but decoding the prediction through the real VAE and scoring the real DINOv2 embedding (so the reward reads exactly the same quantity the projector is trying to approximate, at the cost of a decode and a DINOv2 forward pass every step). The decode-based version shows a real reward effect on top of argmax selection; the projector version, refreshed or not, does not.

A training-time diagnostic. Every reward run also logs an independent monitor: every 100 updates, 32 of the student’s recent predictions are decoded and scored with the real, un-approximated DINOv2 scorer – the quantity the reward is supposed to be increasing, whichever way it is computed. For the decode-based reward this true score rises over training in every one of three seeds ($+0.006$ to $+0.050$). For our projector-reward configuration it does not: across three seeds it fell in two (the largest recent-prediction monitor going from 0.620 to 0.555 over the run) and rose modestly in the third, with no consistent direction. The projector is being optimised – its own score to the student does rise – but the real photographic similarity it is supposed to be a stand-in for is not reliably increasing alongside it. That gap between what the projector reports and what is actually true is the most direct evidence for why refreshing it on the student’s own predictions (Equation 10) does not close the gap to decoding: the refresh keeps P_ϕ agreeing with the true scorer on the 32 predictions it was just refreshed on, but that agreement does not carry forward as the student keeps moving.

3.3 SCALING TO 118,000 CAPTIONS

Selection was re-tested at 118k captions with three training seeds (the reward, in any form, has so far only been tested at the 3k scale above). Table 6 gives the result: $+0.0175$ CompBench ($p = 10^{-14}$) and $+1.29$ GenEval2 over the naive baseline. Under the official ten-images-per-prompt CompBench

Table 4: GenEval2 against argmax alone, 3k pool, three seeds, averaged checkpoints. Overall is the per-prompt score (each prompt’s own geometric mean over its atoms), averaged arithmetically across prompts – the official metric; collapsed is the fraction of prompts with a geometric-mean score under 0.05, i.e. at least one atom entirely wrong. The skill and atom-count rows instead score individual atoms directly (1 if that atom is satisfied, 0 if not, averaged over every atom of that type across all prompts), which is why their scale differs from Overall’s per-prompt aggregate.

Measure	Argmax, no reward	Ours	Δ
Overall (per-prompt mean)	23.7 ± 1.6	23.5 ± 1.6	-0.2 ± 1.2
Collapsed prompts (%)	33.5 ± 1.8	33.7 ± 2.1	$+0.2 \pm 0.5$
Skill: object (per atom)	84.0 ± 0.2	84.1 ± 0.8	$+0.1 \pm 0.8$
Skill: count (per atom)	36.2 ± 1.0	36.5 ± 1.1	$+0.3 \pm 1.7$
Skill: attribute (per atom)	72.0 ± 1.6	72.5 ± 1.4	$+0.5 \pm 0.8$
Skill: position (per atom)	50.8 ± 0.8	51.1 ± 2.0	$+0.3 \pm 2.2$
Skill: verb (per atom)	26.9 ± 1.1	24.6 ± 1.0	-2.3 ± 1.6 ($p=0.12$)
Atoms = 3	72.5 ± 0.6	73.1 ± 1.1	$+0.6 \pm 0.9$
Atoms = 4	72.8 ± 0.6	72.0 ± 1.2	-0.7 ± 1.3
Atoms = 5	66.2 ± 1.5	66.8 ± 1.8	$+0.7 \pm 1.1$
Atoms = 6	65.2 ± 0.3	65.0 ± 1.9	-0.2 ± 2.2
Atoms = 7	61.0 ± 0.2	60.9 ± 1.0	-0.2 ± 1.1
Atoms = 8	58.7 ± 0.9	58.1 ± 1.6	-0.6 ± 0.9
Atoms = 9	56.9 ± 1.1	58.3 ± 1.2	$+1.4 \pm 1.8$
Atoms = 10	50.7 ± 1.2	51.3 ± 1.6	$+0.6 \pm 1.4$

Table 5: Reward source, argmax selection fixed, 3k pool, averaged checkpoints. Δ is against argmax alone (three seeds each, seed-paired).

Reward source	CompBench	Δ vs. argmax alone
None (argmax alone)	0.4868 – 0.4873	–
Projector, frozen after pretraining	0.4847	-0.0026 ± 0.0050 ($p=0.47$)
Projector, refreshed every 100 (default 4 steps)	0.4876	$+0.0003 \pm 0.0075$ ($p=0.95$)
Projector, refreshed every 100, 16 steps (ours)	0.4887	$+0.0014 \pm 0.0035$ ($p=0.57$)
Decode + real DINOv2 (no projector)	0.4923	$+0.0055 \pm 0.0021$ ($p=0.004$)

protocol, the far less noisy evaluation, the gain is $+0.0131$ at $p = 2 \times 10^{-41}$ (Table 7), and it holds in every one of the eight categories under both protocols. The scaled-up student closes most of the gap to its own 28-step teacher: the averaged patch student is 0.021 CompBench below the 28-step teacher and 0.030 above its 8-step source.

3.4 FIDELITY

Selection improves fidelity to real photographs; the reward on top of it does not. Table 8 gives 3k-pool fidelity for the three recipes of Table 2, on the same three seeds throughout for a fair comparison. Argmax selection alone lowers CMMD from 0.837 to 0.800 against the naive baseline. Adding the projector reward moves every one of the four fidelity numbers the wrong way relative to argmax alone – CMMD up, precision down, FID up – though recall is flat. None of these reversals is large against the naive baseline, but they are consistently in the direction of *worse*, not better, which is the opposite of what a working photographic reward should do. Table 9 gives the 118k-scale result for selection alone (the reward has not been tested at that scale): CMMD, precision and recall all improve on every one of the three seeds, and FID is unchanged within its split-half noise.

3.5 QUALITATIVE RESULTS

Figure 1 shows two T2I-CompBench 2D-spatial prompts where our full pipeline (argmax selection with the projector reward) and the frozen 28-step guided teacher differ, illustrating what argmax

Table 6: 118k captions, 56,974 updates, three training seeds, weight-averaged students.

Recipe	CompBench (mean, Δ , p)	GenEval2 (mean, Δ , p)
Random selection (baseline)	0.4668	20.29
Argmax selection (ours)	0.4843, +0.0175, 10^{-14}	21.58, +1.29, 0.02

Table 7: Official ten-images-per-prompt T2I-CompBench, 118k weight-averaged students, three seeds.

Recipe	CompBench (mean, Δ , p)
Random selection (baseline)	0.4719
Argmax selection (ours)	0.4850, +0.0131, 2×10^{-41}

selection buys overall (Table 2’s naive-vs-ours gap, which the reward does not meaningfully add to – Section 3.2), not a claim specific to the reward. Each prompt names two objects and a specific left/right or top/bottom relation between them; we generated our student at its trained 4-step budget and, to check the placement was not a 4-step accident, again at 8 and 28 steps, and compared against the frozen 28-step teacher sampled with classifier-free guidance ($w=7$), all four images per prompt from identical initial noise. In both prompts every one of our three student samples places the two named objects in the relation the prompt asks for; the guided teacher does not, placing them the other way around (“a boy on the left of a balloon”) or without the second object resting on the first (“a candle on the top of a chicken”). These two prompts were found by inspecting images, not by scores alone: several other high-scoring-gap prompts we checked turned out to be genuinely correct in both the student’s and the teacher’s image, with the automatic evaluator’s score gap coming from image framing rather than a visible placement error, and are not shown here. At its trained 4-step budget the student’s automatic CompBench score is 0.92 against the teacher’s 0.00 on the first prompt and 0.78 against 0.00 on the second; we did not re-run the automatic evaluator on the 8-step and 28-step student images, so only the 4-step column carries a formal score. The broader point is qualitative: a distilled few-step student is not, skill by skill, a strictly degraded copy of its many-step guided teacher.

4 ADDITIONAL ABLATIONS

The following ablations were run on the 3k pool and are reported in full in the appendix; only their titles are given here.

- Selection rule: argmax vs. Boltzmann-weighted vs. sampled vs. uniform candidate weighting.
- Projector refresh schedule: frozen vs. refreshed every 100 updates at the default 4 steps vs. 16 steps (ours) vs. refreshed every 25 updates.
- The decode-based reward’s own reward-weight and rewarded-state sweep, as the point of comparison for Section 3.2 (that sweep was run for the decode-based reward, not the projector, which was never tuned past the refresh schedule above).
- Consistency-target horizon: 1-step Tweedie estimate vs. a 2-step chord vs. the trajectory endpoint.
- Training objective: x_0 regression vs. segment-velocity imitation.
- Teacher quality: candidate generation and training with a 16-step and a 28-step teacher.
- Regression onto the reference photograph as a second training target.
- An evolving (exponential-moving-average) teacher in place of the frozen teacher.
- Number of candidates ($N=4$ vs. $N=8$) and optimiser batch size, at the 118k scale.
- Inference step count: sampling the 4-step student at 2, 8, 16 and 28 steps.

Table 8: Fidelity, 3k pool, three matched seeds (0–2), averaged checkpoints, 5,000 COCO val2017 captions.

Recipe	CMMD ↓	Precision ↑	Recall ↑	FID ↓
Random selection, no reward	0.837 ± 0.042	0.479 ± 0.025	0.054 ± 0.003	30.42 ± 0.17
Argmax selection, no reward	0.800 ± 0.030	0.489 ± 0.006	0.066 ± 0.004	30.12 ± 0.36
Argmax selection + projector reward (ours)	0.817 ± 0.025	0.474 ± 0.013	0.067 ± 0.001	30.89 ± 0.22

Table 9: Fidelity at 118k, weight-averaged students, three seeds (s0 / s1 / s2), 5,000 COCO val2017 captions. The 28-step teacher is shown for reference, not as a fair comparison at 4 steps.

Recipe	FID ↓	CMMD ↓	Precision ↑	Recall ↑
Random (baseline)	31.08 / 30.97 / 31.44	0.84 / 0.84 / 0.83	0.485 / 0.489 / 0.503	0.059 / 0.066 / 0.062
Argmax selection (ours)	30.98 / 30.83 / 31.47	0.78 / 0.79 / 0.78	0.522 / 0.511 / 0.522	0.071 / 0.068 / 0.063
Teacher, 28 steps, $w=7$	29.21	0.64	0.549	0.156

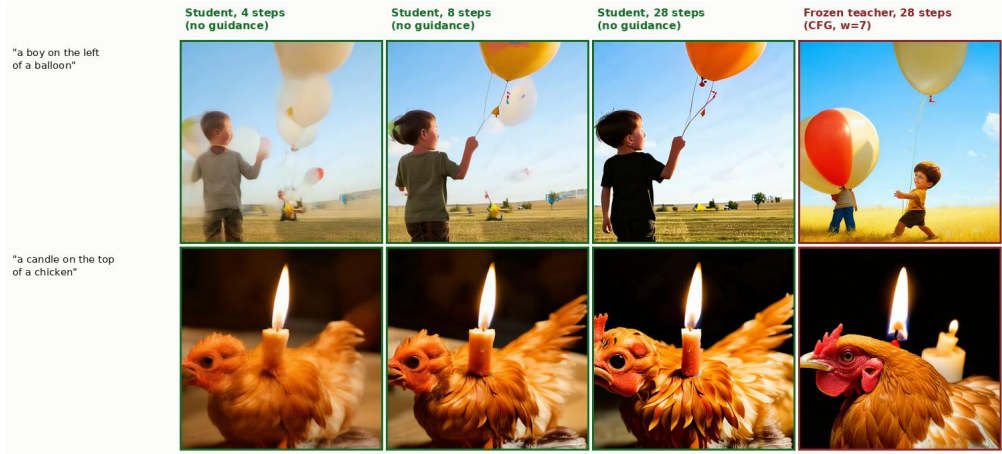


Figure 1: Two T2I-CompBench 2D-spatial prompts, identical initial noise per prompt (columns are not independently seeded). Our student (argmax selection + projector reward, no guidance) places both named objects in the requested relation at 4, 8 and 28 sampling steps alike; the frozen teacher with guidance ($w=7$) does not, at 28 steps. Green borders: the requested relation is satisfied. Red border: it is not. Automatic CompBench 2D-spatial scores (only computed for the trained 4-step student and the teacher): 0.92 vs. 0.00 (top row), 0.78 vs. 0.00 (bottom row).